ManiGit: Transparent, Versioned Context Memory for Long-Running AI Agents

Techgium Hackathon | Problem Statement 54

The ManiGit project aims to provide context memory for long-running AI agents in a transparent and versioned manner.

Problem

AI agents can have thousands of messages over many sessions. It will delay sending to the model and will be expensive; and it will reduce the quality of answers as important facts are lost. Others are more frequently employed such as the sliding windows or blanket summarization; these do not capture the salient information and the user does not know what was not captured or is wrong. The common outcome is to do the thing again.

Core Concept

The agent memory is a version controlled repository in ManiGit. All raw messages are permanently saved in an append only archive. The working context is a managed view of this archive (to which the model is accessing) and each view change is committed. There is no hiding—not anywhere—and no one can snatch anything from her.

Solution

Automatic three-tier decisions. They are sorted into keep verbatim (constraints, decisions, exact numbers, IDs, code, error messages), summarize (reasoning and discussion; connected to the originals), or discard from context (duplicates, superseded outputs, small talk, retained as recoverable tombstones). Any model can be used for the system since it is built around a model-agnostic LLM judge (with a swappable adapter) comprising speedy rules.

Chain of memory with version. Each step is a commit having a topic and it includes the information of the operation applied, the reasons behind the operation and number of tokens before and after it.

Several Temporalities and Branching. Users can scroll through what is currently in the store, replay the agent's knowledge of the past, navigate back to the previous commit, branch to explore a different route and then merge branches again, but this time with the user being confronted with any conflicts.

Human control & human explanations. Summaries may be pin, trim, expand or hand edited – all hand edits are commits. Why report includes a statement of what is saved, what is discarded, and why.

Budget-aware assembly. Context is built in a token budget, and begins with pinned facts, then relevant topic summaries, then recent verbatim window, then anything the user expands. The use of topical retrieval by label does not allow for the increase of cost with length of history.

Innovation

ManiGit does not only allow auditing of every memory decision, but also reversibility, which is not offered in opaque summarizers. Version-control semantics (checkout, branch, merge) and tiered compression enables agents to add memory, which users can view, fix, and direct.

Evaluation and Expected Impact

All baselines will be compared in the context of long multi-session conversations, using token-per-request, cost, latency, and accuracy of answers (as well as accuracy of recalled earlier facts and/or revised decisions). The objective is to reduce the number of tokens, the cost and the response time, and/or to increase the quality of answer. Memory can be checked and patched anytime, without requiring a user to restart the agent.

Tech Stack

Python, SQLite, a model adapter, inspired by LiteLLM, a CLI and a Web UI, which displays the commit graph with tier coloured context.